UNIVERSIDAD MARIANO GÁLVEZ

SEDE: CHIMALTENANGO, COLEGIO MESOAMERICANO

FACTULTAD DE INGENIERÍA EN SISTEMAS

CURSO: 029 – SISTEMAS OPERATIVOS I

ING. JOSUÉ GIMÉNEZ

SYSMONITOR WEB - MANUAL TÉCNICO

NOMBRE:	CARNÉ
OSCAR EDUARDO GIRÓN MORALES	1990-24-12952
PABLO MOISÉS HERNÁNDEZ VICENTE	1990-24-12399
JORDY ALEJANDRO NAJARRO CHEREC	1990-24-11866
MARIO GIOVANNY MALDONADO MADRID	1990-24-3894
JOSÉ NEHEMÍAS PICHIYÁ APÉN	1990-24-25116
ALLAN ANTONY AJUCHÁN POPOL	1990-24-13391

FECHA: 31 DE OCTUBRE DE 2026

SysMonitor Web — Manual técnico

1. Introducción

SysMonitor Web es una aplicación web Laravel que permite observar y administrar, desde el navegador, el sistema operativo Linux (Debian 12) donde está instalada. Obtiene datos reales del kernel leyendo /proc y ejecuta comandos de una lista blanca, sin intérprete de comandos. Fue desarrollada para el curso Sistemas Operativos 1 (código 029) de la Universidad Mariano Gálvez de Guatemala.

URL pública: https://www.sysmonitorwebumg.com/.

Repositorio: https://github.com/oscargiron2522/app_sistemas_oper0ativosI.

Alcance: monitoreo de procesos, CPU, memoria, almacenamiento y usuarios; administración de procesos de prueba (señales y renice); simuladores de planificación de CPU e interbloqueos; autenticación por roles y bitácora de acciones.

Componente	Tecnología
Sistema operativo	Debian 12 (Bookworm), VPS en producción
Servidor web	Apache 2 (VirtualHost con DocumentRoot en public/)
Backend	PHP 8.2 o superior con Laravel 12 (extensiones sqlite3, mbstring, xml, posix)
Base de datos	SQLite (database/database.sqlite) con migraciones de Laravel
Frontend	Blade, Bootstrap 5.3.3 y Bootstrap Icons 1.11.3 (CDN), CSS propio public/css/linuxcore.css, JavaScript propio public/js/linuxcore.js
Gráficas	Chart.js 4.4.4 (CDN), actualizadas con fetch
Seguridad de transporte	HTTPS con Let's Encrypt (certbot --apache)
Control de versiones	Git y GitHub

2. Arquitectura general

La aplicación sigue el patrón MVC de Laravel. No hay una capa de servicios separada: cada controlador de módulo lee /proc o ejecuta comandos de la lista blanca con sus propios métodos auxiliares. Flujo de una petición:

1. El navegador solicita una URL por HTTPS; Apache la recibe en el VirtualHost y la entrega a PHP (public/index.php).
2. Laravel resuelve la ruta en routes/web.php y aplica los middleware: guest para el login, auth para el resto y throttle:10,1 en el envío del login.
3. El controlador valida la entrada con las reglas de Laravel (PID entero, señal de una lista fija, rangos numéricos).
4. Las acciones administrativas llaman a exigirAdmin(): un Observador recibe 403 y el intento queda en la bitácora.
5. El controlador lee /proc o ejecuta un comando permitido, y registra la acción con Bitacora::registrar() (usuario, fecha y hora, acción, objetivo, resultado e IP).
6. Devuelve una vista Blade o JSON. Las pantallas piden datos con fetch a rutas JSON (por ejemplo /cpu-memoria/datos y /procesos/datos) y Chart.js dibuja las gráficas.

Capas:

- Presentación: vistas Blade (resources/views), CSS y JavaScript propios, Bootstrap y Chart.js.
- Aplicación: rutas, middleware y diez controladores en app/Http/Controllers.
- Datos: cuatro modelos Eloquent sobre SQLite (User, Bitacora, ProcesoPrueba, Simulacion).
- Sistema operativo: kernel Linux, que expone la información en /proc.

3. Estructura de carpetas

Estructura real del repositorio (se omiten vendor, node_modules y los archivos estándar de Laravel que no se modificaron):

```
app_sistemas_operativosI/
├── app/
│   ├── Http/Controllers/
│   │   ├── AuthController.php          # login y logout
│   │   ├── DashboardController.php     # inicio
│   │   ├── ProcesosController.php      # M1
│   │   ├── CpuMemoriaController.php    # M2
│   │   ├── PlanificacionController.php # M3
│   │   ├── InterbloqueosController.php # M4
│   │   ├── AlmacenamientoController.php # M5: discos y explorador
│   │   ├── UsuariosController.php      # M5: usuarios, grupos y sesiones
│   │   └── BitacoraController.php      # M6
│   └── Models/                         # User, Bitacora, ProcesoPrueba, Simulacion
├── config/
│   └── sysmonitor.php                  # rutas del explorador y máximo de procesos de prueba
├── database/
│   ├── migrations/                     # users, bitacora, procesos_prueba, simulaciones
│   ├── seeders/DatabaseSeeder.php      # cuentas admin y observador
```

```
│   └── database.sqlite              # se crea en la instalación
├── deploy/
│   └── sysmonitor.conf              # plantilla del VirtualHost de Apache
├── public/                          # DocumentRoot de Apache
│   ├── css/linuxcore.css
│   └── js/linuxcore.js
├── resources/views/                 # layouts/app, auth/login, dashboard,
                                       procesos,
│                                    # cpu, planificacion, interbloqueos,
                                       almacenamiento,
│                                    # explorador, usuarios, bitacora
├── routes/web.php                   # todas las rutas
├── Inter_Grafica_Modu/              # bocetos (mockups) de la interfaz
├── .env.example                     # plantilla; el .env real nunca se sube
└── README.md
```

4. Rutas de la aplicación

Rutas definidas en routes/web.php. Solo el login es público; todas las demás exigen sesión (middleware auth). Lanzar procesos, enviar señales y cambiar prioridad exigen además el rol Administrador (un Observador recibe 403). No existe ruta de registro de usuarios.

Módulo	Método y ruta	Rol	Función
Acceso	GET /login, POST /login	Público (guest)	Formulario y autenticación; el envío admite 10 intentos por minuto
Acceso	POST /logout	Autenticado	Cierra la sesión
Inicio	GET /	Admin y Observador	Dashboard
M1	GET /procesos	Admin y Observador	Pantalla de procesos
M1	GET /procesos/datos	Admin y Observador	JSON con la lista de procesos y el resumen por estado
M1	GET /procesos/top	Admin y Observador	JSON con el top 5 por CPU y por memoria
M1	GET /procesos/pruebas	Admin y Observador	JSON con los procesos de prueba vivos

Módulo	Método y ruta	Rol	Función
M1	GET /procesos/{pid}	Admin y Observador	Detalle de /proc/PID/status (pid numérico)
M1	POST /procesos/lanzar	Solo Admin	Lanza un proceso de prueba (sleep o script de carga)
M1	POST /procesos/senal	Solo Admin	SIGTERM, SIGKILL, SIGSTOP o SIGCONT a un proceso de prueba
M1	POST /procesos/renice	Solo Admin	Cambia el nice de un proceso de prueba
M2	GET /cpu-memoria	Admin y Observador	Pantalla de CPU y memoria
M2	GET /cpu-memoria/datos	Admin y Observador	JSON que se consulta cada pocos segundos
M3	GET /planificacion	Admin y Observador	Simulador
M3	POST /planificacion/simular	Admin y Observador	Ejecuta y guarda la simulación
M3	GET /planificacion/historial	Admin y Observador	Últimas 20 simulaciones guardadas
M3	GET /planificacion/{id}	Admin y Observador	Recupera una simulación guardada
M4	GET /interbloqueos	Admin y Observador	Simulador de interbloqueos
M4	POST /interbloqueos/banquero	Admin y Observador	Algoritmo del banquero y evaluación de solicitud
M4	POST /interbloqueos/deteccion	Admin y Observador	Detección de interbloqueos, grafo y recuperación
M5	GET /almacenamiento	Admin y Observador	Discos, particiones, df, findmnt y fstab

Módulo	Método y ruta	Rol	Función
M5	GET /explorador	Admin y Observador	Explorador de solo lectura (parámetro ruta)
M5	GET /usuarios	Admin y Observador	Usuarios, grupos y sesiones activas
M6	GET /bitacora	Admin y Observador	Bitácora paginada de 15 en 15
Sistema	GET /up	Público	Comprobación de salud de Laravel

5. Controladores y modelos

Módulo	Controlador	Métodos principales	Responsabilidad
Acceso	AuthController	mostrarLogin, login, logout	Autentica con usuario y contraseña, regenera la sesión y registra inicios y cierres de sesión, incluidos los fallidos
Inicio	DashboardController	index	Muestra el dashboard; los indicadores se refrescan con /cpu-memoria/datos
M1	ProcesosController	index, datos, top, detalle, pruebas, lanzar, senal, renice	Lee /proc, calcula el resumen por estado y el top 5, lanza procesos de prueba y aplica señales y renice con validación; helpers exigirAdmin, verificarPrueba, leerProceso y leerStatBasico
M2	CpuMemoriaController	index, datos	Calcula el uso de CPU con dos lecturas de /proc/stat y lee memoria, swap, carga y uptime; helpers infoCpu, usoCpu, leerMeminfo y metricas
M3	PlanificacionController	index, simular, historial, ver	Implementa FCFS, SJF, SRTF, Round Robin, Prioridades no expropiativo y Prioridades expropiativo; calcula Gantt, esperas, retornos, respuesta y utilización; guarda cada simulación

Módulo	Controlador	Métodos principales	Responsabilidad
M4	InterbloqueosController	index, banquero, deteccion	Calcula la matriz Necesidad, el estado seguro y la secuencia segura con paso a paso, evalúa solicitudes, detecta interbloqueos, busca ciclos en el grafo (DFS) y propone recuperación por terminación o apropiación
M5	AlmacenamientoController	index, explorador	Ejecuta lsblk, df y findmnt de una lista blanca; lee /etc/fstab y /proc/swaps; navega directorios permitidos en solo lectura
M5	UsuariosController	index	Lee /etc/passwd y /etc/group (distingue usuarios humanos y de sistema) y ejecuta who
M6	BitacoraController	index	Lista la bitácora, la más reciente primero, paginada

Modelos (app/Models): User (método esAdmin; rol admin u observador), Bitacora (método estático registrar, que guarda usuario, acción, objetivo, resultado e IP), ProcesoPrueba (registro de los procesos que lanzó la aplicación) y Simulacion (guarda procesos y resultado como JSON). Los siete controladores de módulos no usan una capa de servicios; los helpers son métodos privados o estáticos de cada clase.

6. Modelo de datos

La base es un archivo SQLite (database/database.sqlite) creado con cuatro migraciones de Laravel. Las columnas user_id no tienen llave foránea declarada; son enteros que apuntan a users.id y pueden ser nulos. La sesión y la caché usan archivos (SESSION_DRIVER=file, CACHE_STORE=file), por lo que no hay tablas para ellas.

users (0001_01_01_000000_create_users_table)

Columna	Tipo	Descripción
Id	integer, PK	Identificador
Name	string	Nombre visible
Usuario	string, único	Nombre de inicio de sesión

Columna	Tipo	Descripción
Password	string	Contraseña con hash (bcrypt)
Rol	string, por defecto observador	admin u observador
remember_token	string, nulo	Token de Laravel
created_at, updated_at	timestamp	Control de fechas

bitacora (2025_01_01_000001_create_bitacora_table)

Columna	Tipo	Descripción
Id	integer, PK	Identificador
user_id	integer, nulo	Usuario que ejecutó la acción
Usuario	string, nulo	Nombre de usuario (también se guarda en intentos fallidos)
Accion	string	Acción realizada, por ejemplo Señal SIGTERM
Objetivo	string, nulo	Proceso afectado, por ejemplo PID 1234 (sleep 300)
Resultado	string, nulo	Éxito o error
Ip	string(45), nulo	Dirección IP de origen
created_at, updated_at	timestamp	Fecha y hora de la acción

procesos_prueba (2025_01_01_000002_create_procesos_prueba_table)

Columna	Tipo	Descripción
Id	integer, PK	Identificador
Pid	unsigned integer	PID del proceso lanzado por la aplicación
inicio_proc	string	Campo starttime de /proc/PID/stat; evita confundir un PID reutilizado
Commando	string	Etiqueta, por ejemplo sleep 300

Columna	Tipo	Descripción
user_id	integer, nulo	Administrador que lo lanzó
created_at, updated_at	timestamp	Fechas

simulaciones (2025_01_01_000003_create_simulaciones_table)

Columna	Tipo	Descripción
Id	integer, PK	Identificador
user_id	integer, nulo	Usuario que simuló
Algoritmo	string	FCFS, SJF, SRTF, RR, PRIO o PRIO_EXP
Quantum	unsigned integer, nulo	Solo para Round Robin
Procesos	json	Llegada, ráfaga y prioridad de cada proceso
Resultado	json	Gantt, tiempos por proceso, promedios y comparativa
espera_promedio, retorno_promedio	decimal(10,2)	Promedios
Utilizacion	decimal(6,2)	Utilización de CPU en porcentaje
created_at, updated_at	timestamp	Fechas

Relaciones: un usuario tiene muchas entradas de bitácora, muchos procesos de prueba y muchas simulaciones. El seeder (DatabaseSeeder) crea las cuentas admin y observador; sus contraseñas salen de ADMIN_PASSWORD y OBSERVADOR_PASSWORD en el .env, o se generan al azar y se muestran en la consola.

7. Archivos de /proc y comandos por módulo

Módulo	Fuente de datos	Qué se obtiene
M1 Procesos	Directorios numéricos de /proc, /proc/PID/stat,	PID, PPID, estado (R, S, D, Z, T), nice, hilos, tiempo de CPU

Módulo	Fuente de datos	Qué se obtiene
	/proc/PID/status, /proc/PID/cmdline	(utime + stime), inicio, memoria residente (VmRSS) y comando
M1 Procesos	/etc/passwd, /proc/meminfo, /proc/uptime	Nombre del usuario por UID, porcentaje de memoria y % CPU acumulado, como ps
M1 Procesos	Dos lecturas de ticks separadas 0,4 s	Top 5 por CPU; el top 5 por memoria sale de VmRSS
M1 Procesos	exec con comando fijo: nohup sleep N o un bucle de carga	Lanzar procesos de prueba; N es un entero validado entre 10 y 3600
M1 Procesos	posix_kill con SIGTERM (15), SIGKILL (9), SIGSTOP (19), SIGCONT (18)	Señales a procesos de prueba
M1 Procesos	/usr/bin/renice -n N -p PID (proc_open con arreglo, sin shell)	Cambio de prioridad, N entre 0 y 19
M2 CPU y Memoria	/proc/cpuinfo, /proc/uptime	Modelo, núcleos, frecuencia y tiempo encendido
M2 CPU y Memoria	/proc/stat (dos lecturas separadas 0,3 s)	Porcentaje de uso de CPU
M2 CPU y Memoria	/proc/loadavg, /proc/meminfo	Carga a 1, 5 y 15 minutos; memoria total, usada, caché, libre, disponible y swap
M3 y M4	Sin comandos del sistema	Simulaciones calculadas en PHP; los datos los escribe el usuario
M5 Almacenamiento	lsblk -b -J -o NAME,TYPE,SIZE,FSTYPE,MOUNTPOINT,PTTYPE,MODEL,RO	Discos, particiones, tipo de tabla (GPT o MBR) y punto de montaje
M5 Almacenamiento	df -PT -B1	Tamaño, usado, libre y porcentaje por sistema de archivos

Módulo	Fuente de datos	Qué se obtiene
M5 Almacenamiento	findmnt -J -o TARGET,SOURCE,FSTYPE,OPTIONS (respaldo: /proc/mounts), /etc/fstab, /proc/swaps	Puntos de montaje, fstab y swap
M5 Usuarios	/etc/passwd, /etc/group, comando who	Usuarios (humano si UID desde 1000 y menor a 65534; los demás, de sistema), grupos y sesiones activas
M5 Explorador	scandir y lstat dentro de los directorios de SYSMON_RUTAS	Permisos rwx, propietario, grupo, tamaño y fecha, solo lectura

Porcentaje de uso de CPU entre dos lecturas de /proc/stat (el tiempo inactivo suma idle e iowait):

\text{uso} = \left(1 - \frac{\Delta\text{inactivo}}{\Delta\text{total}}\right) \times 100
Relación con los conceptos del curso: los estados de M1 corresponden al modelo de estados de un proceso (en ejecución, dormido, espera de E/S, zombie y detenido) y las señales ilustran su control; el nice muestra la prioridad de planificación; M2 muestra memoria física, caché y swap; M3 aplica algoritmos de planificación; M4 aplica el algoritmo del banquero y la detección de interbloqueos; M5 muestra tablas de particiones, sistemas de archivos, permisos y usuarios.

8. Medidas de seguridad

Riesgo	Medida implementada
Inyección de comandos	Ningún dato del usuario se concatena en un comando. El PID es un entero validado por Laravel (mínimo 2), la señal debe estar en una lista fija y el nice es un entero entre 0 y 19. Los comandos con argumentos (renice, lsblk, df, findmnt, who) se ejecutan con proc_open y un arreglo de argumentos, sin shell, por lo que no hace falta escapeshellarg. Las señales usan posix_kill, sin comando.
Comandos arbitrarios	Lista blanca: lsblk, df y findmnt están en una constante de AlmacenamientoController; renice y who tienen rutas fijas. Los procesos

Riesgo	Medida implementada
	de prueba se lanzan con dos comandos fijos; lo único variable es un entero de 10 a 3600.
Escalada de privilegios	Solo se actúa sobre procesos de prueba registrados por la aplicación (tabla procesos_prueba). Se compara el starttime guardado con el actual para no afectar a un PID reutilizado, se exige que el proceso pertenezca al usuario de la aplicación (www-data) y hay un máximo de 10 procesos de prueba. No se otorga sudo a www-data.
Acceso no autorizado	Todas las rutas, salvo el login, exigen sesión. No hay registro de usuarios. El envío del login admite 10 intentos por minuto, se regenera la sesión al entrar y las contraseñas se guardan con hash bcrypt.
Acciones sin permiso	Lanzar, señal y renice llaman a exigirAdmin(): el Observador recibe 403 y el intento se registra como Acción denegada.
Falta de trazabilidad	Bitacora::registrar() guarda usuario, fecha y hora, acción, proceso afectado, resultado e IP en cada inicio de sesión, acción administrativa y denegación.
Exploración del sistema de archivos	El explorador es de solo lectura y solo muestra directorios dentro de SYSMON_RUTAS (por defecto /var/log, /etc, /home, /tmp, /var/www y /opt). Resuelve la ruta con realpath, rechaza bytes nulos y no sigue enlaces que salgan de las rutas permitidas.
Formularios	Protección CSRF de Laravel; las peticiones fetch envían el token.
Credenciales expuestas	El .env y los archivos .sqlite están en .gitignore. Las contraseñas de demostración salen del .env o se generan al azar al ejecutar el seeder.
Exposición de red	Firewall ufw con solo los puertos 22, 80 y 443; SSH con llave y sin login de root; HTTPS

Riesgo	Medida implementada
	obligatorio con redirección desde HTTP; APP_DEBUG=false en producción.